



Geminifs分析

由 彭昆仑创建, 最后修改于大约1分钟以前



- 整体介绍
 - Geminifs是什么
 - 背景：GPU从磁盘读写数据的技术路径演化
 - 上层应用使用Geminifs的方式
 - Geminifs代码目录
 - Geminifs组件的整体层次图
- Geminifs的实现原理
 - 背景知识：NVME SSD的相关原理
 - GPU和CPU共享NVME SSD
 - PCIE P2P以及NVME SSD地址、内存地址、显存地址之间的映射
 - 对文件的管理以及文件元数据的使用
- Geminifs的具体实现
 - 整体流程
 - 创建和使用的设备文件
 - 类图
 - 具体运行过程
 - 先加载自定义内核驱动模块
 - 再运行应用程序
 - TestForGeminiFS的整体逻辑
 - Geminifs的构造函数
 - Controller的构造函数
- Geminifs和VLLM的结合

▼ 整体介绍

▼ Geminifs是什么

Geminifs是一个面向AI场景访存需求的高性能GPU文件系统，支持GPU在向nvme ssd发起文件读写请求时绕开CPU和内存拷贝以减少性能损耗，是在bam的基础上，通过让GPU和CPU共享文件系统的元数据、共享nvme ssd，来实现GPU可以直接读写nvme ssd上的文件数据。Geminifs于今年2月发表在顶会USENIX上。

Geminifs有如下特点：

- 1)GPU-Centric，允许在GPU进程直接提交IO请求，完全绕过CPU，最小化软件开销
- 2)提供文件系统管理，实现磁盘空间管理能力
- 3)向CPU和GPU提供统一命名空间，CPU和GPU都可以打开并直接读写文件
- 4)Geminifs利用GPU线程并发读写文件切片，并发数远比传统的CPU读写文件大，这也是Geminifs读写文件性能高于传统的CPU先读写文件再进行内存和显存间数据拷贝的方式的原因。

▼ 背景：GPU从磁盘读写数据的技术路径演化

GPU从磁盘读写数据的技术路线，按照是否需要CPU参与，可分为CPU-Centric和GPU-Centric，如下图。

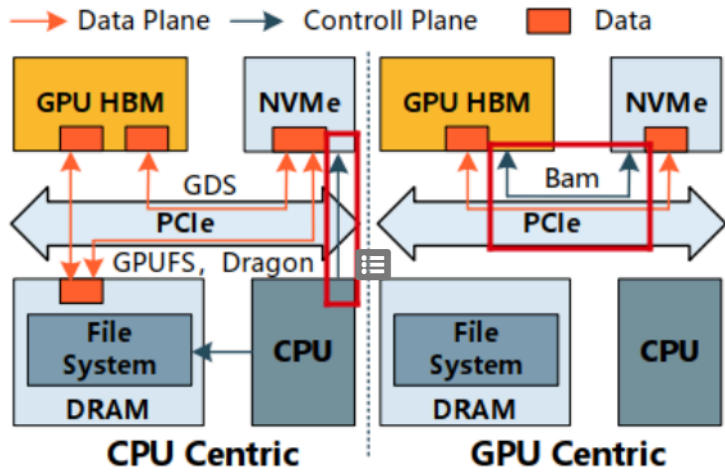


图1.CPU-Centric和GPU-Centric架构图

1.CPU-Centric：访问存储数据的过程中，需要CPU参与

1.1) cpu先把硬盘数据读到内存，然后拷贝到GPU显存（通过cudaMemcpy）

1.2) GPUfs、Dragon等提供了Posix-like的GPU核函数，允许在GPU进程内直接提交文件系统请求，但实际IO仍通过CPU提交

1.3) GDS（GPU Direct Storage）：nvidia提出的GPU显存和存储直接DMA，但IO请求还是需要由CPU发起和初始化。

GDS将GPU buffer的DMA地址注册到主机内存中（cuFileBufRegister），建立GPU buffer与主机phony buffer的映射；GDS使用phony buffer提交IO请求；在NVMe层进行DMA地址替换，从而实现NVMe设备和GPU HBM的peer-to-peer 直通。

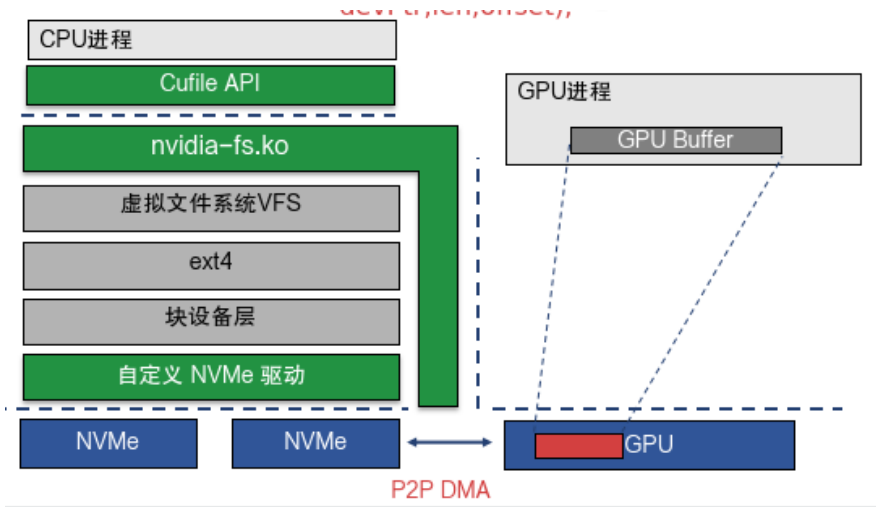


图2.GDS架构图

2.GPU-Centric:访问存储数据的过程中，不需要CPU参与

2.1) Bam:通过将nvme队列和门铃寄存器映射到GPU显存，使得控制平面和数据平面都由GPU进程管理，让GPU可以直接发起IO请求，完全bypass cpu。

但是Bam的不足：

- a) GPU以块设备管理的方式使用ssd裸盘，没有文件系统层次的管理
- b) 在访问主机文件系统管理的数据时，仍然需要内存拷贝，因为文件系统的元数据在CPU内存
- c) GPU独占nvme设备，设备无法共享

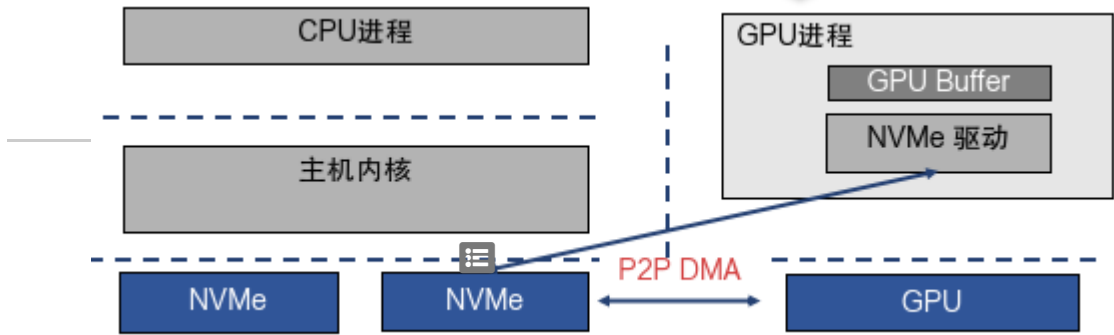


图3.Bam整体架构图

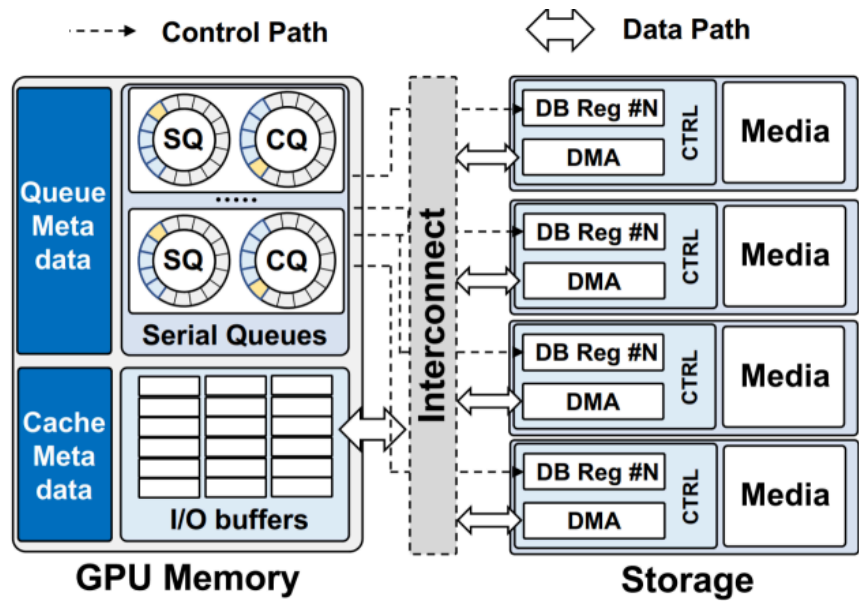


Figure 1: Logical view of BaM design.

图4.Bam具体架构图

2.2) Geminifs

在Bam的基础上，通过让CPU和GPU共享NVME SSD、GPU读主机文件系统的元数据等方式，比Bam多出了下列优点：

- a)让GPU也能读写ssd的文件，而不是像Bam那样，只能把ssd当块设备使用。
- b)当基于Bam读写主机文件时，仍然需要内存拷贝，而基于Geminifs则不需要内存拷贝。

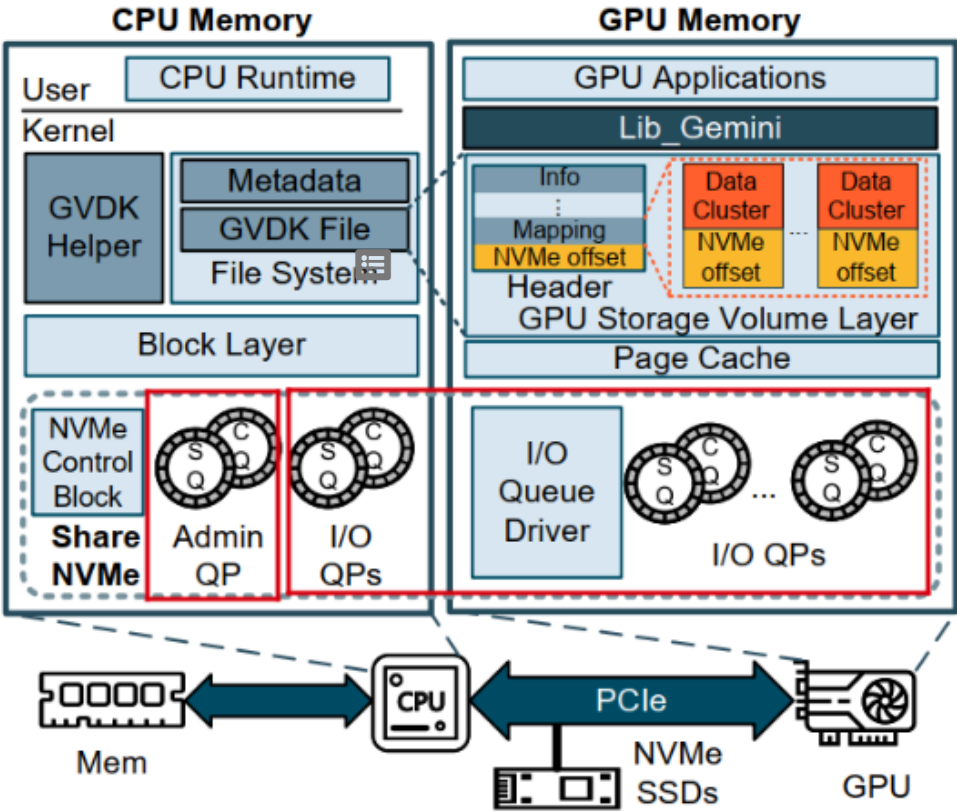


图5.Geminifs架构图

上层应用使用Geminifs的方式

在Geminifs项目中，封装了Geminifs类，通过该类对外提供了Geminifs的API，上层应用程序可以基于Geminifs类，搭建项目。

Geminifs类对外提供的部分方法	方法的作用	部分入参	备注
GeminiFS构造函数	实例化Geminifs	配置文件的路径、GPUFile的数量	
geminifs_gpu_open_file	打开一个GPUFile，获取对应的维护信息	device_id, gpu_file_id	
geminifs_gpu_close_file	关闭一个GPUFile，释放相关资源	device_id, gpu_file_id	
geminifs_register_tensor_with_gpu	对tensor切片，记录每个切片的物理显存地址，并建立DMA映射，用于在后续GPU向nvme ssd发IO命令时，让ssd通过DMA读写对应切片数据	tensor	需要显卡支持PCIE P2P，会在geminifs内核模块中调显卡驱动提供的函数
geminifs_GPU_write_kernel	GPU向指定GPUFile写入tensor	tensor,gpu_file_id, device_id	会启动核函数

Geminifs类对外提供的部分方法	方法的作用	部分入参	备注
geminifs_GPU_read_kernel	GPU从指定GPUFile读取tensor	tensor,gpu_file_id, device_id	会启动核函数
geminifs_batched_write	GPU向指定GPUFile批量写入tensor	tensor向量, gpu_file_id, device_id	底层实现和geminifs_GPU_write_kernel一样
geminifs_batched_read	GPU从指定GPUFile批量读取tensor	tensor向量, gpu_file_id, device_id	底层实现和geminifs_GPU_read_kernel一样

▼ Geminifs代码目录

目录	作用
examples	示例demo
libgeminifs	编译产物： libgeminifs.so ，负责geminifs文件系统的逻辑，并向应用程序提供API
libnvm	编译产物： libnvm.so ，负责在用户态和snvme内核模块通信
modules	编译产物：自定义内核模块snvme.ko, snvme-core.ko snvme-core.ko：负责创建workqueue，申请设备号，创建设备类,创建所绑定ssd的字符设备文件，例如/dev/snvme0 snvme.ko：负责初始化一些list，创建字符设备文件/dev/snvme_control
scripts	工具脚本

▼ Geminifs组件的整体层次图

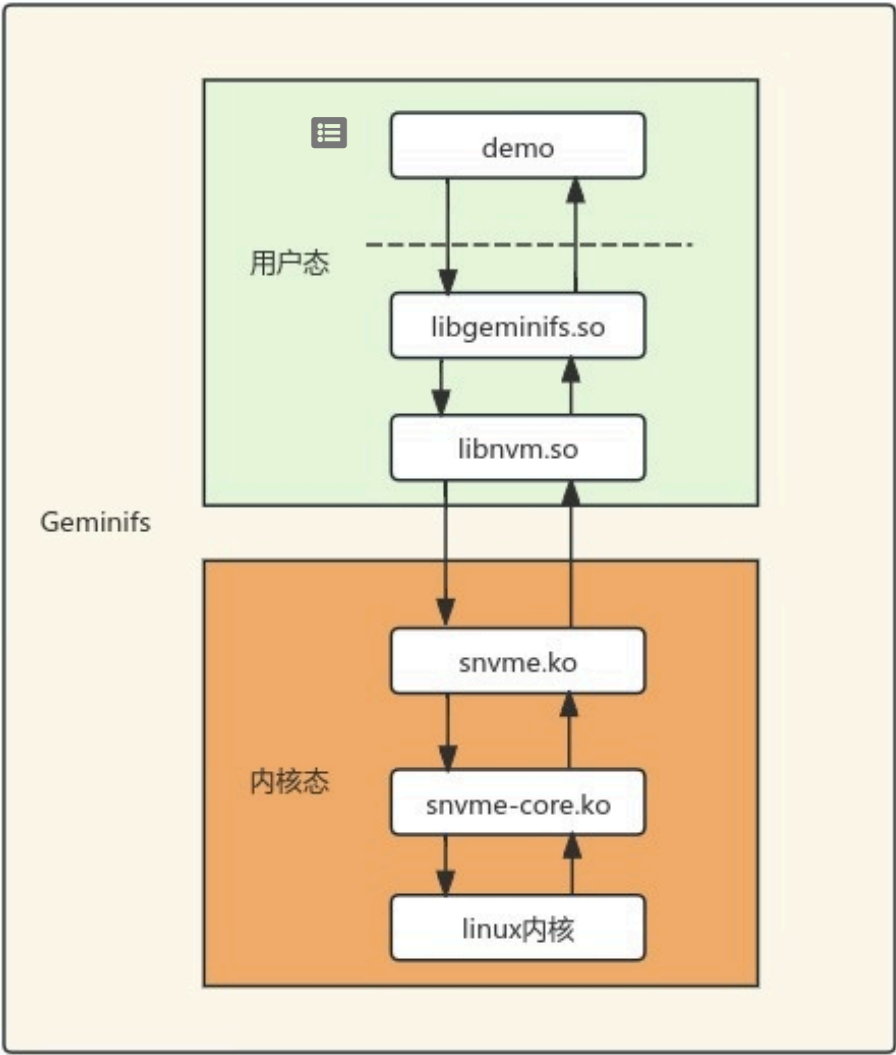


图6.Geminifs的组件整体层次图

▼ Geminifs的实现原理

GPU本身并不适合独立构建文件系统，因为GPU主要针对高并发计算任务进行设计，不适合处理文件系统中复杂的逻辑控制和分支跳转。

Geminifs主在GPU上实现了一个轻量的文件系统，和主机CPU文件系统伴生共存。本节介绍Geminifs的主要技术原理，为了方便描述和理解，会先简要介绍下NVME SSD的相关原理作为铺垫。

▼ 背景知识：NVME SSD的相关原理

1) admin queue和 io queue

NVME SSD通过sq/cq队列和主机交互，队列可分为admin queue和 io queue两种类型，分别对应控制面和数据面，作用分别如下：

- admin queue负责控制面，即对ssd的所有管理操作，如增删io queue、管理namespace、固件管理、错误处理、设置特性等。
- io queue负责数据面，即数据读写命令的通信。

每个ssd只有一对admin queue，会有多对io queue。每个sq和cq都有对应的门铃寄存器，用于主机和ssd的通信。

2) 逻辑块、ssd io命令的内容、PRP



ssd的最小访存单元是逻辑块，主机在指定要读写的ssd数据范围时，是指定ssd的逻辑块号(logic block address, lba)和要读写的逻辑块的数量，ssd内部的控制器再把逻辑块映射为物理块。主机不感知ssd物理块。

主机发往ssd的nvme_cmd有对应的数据格式，里面的内容包括：

- cid (命令号)
- opcode (表示读写操作类型)
- prp1,prp2 (表示物理地址)
- starting_lba(读写ssd的起始块号)、要读写的ssd块数

其中，PRP(physical region page)是NVME协议里的术语，表示ssd要通过DMA读写的物理地址，在传统的CPU读写ssd中，prp即是物理内存地址：

- 在cpu写ssd时，prp指示ssd要去读取的缓冲区的物理内存地址，即ssd从prp对应物理内存地址读数据，写入starting_lba起始的逻辑块
- 在cpu读ssd时，prp指示ssd要去写的缓冲区的物理内存地址，即ssd从starting_lba起始的逻辑块读数据，写入prp对应物理内存地址

3) 主机和ssd通信时，对sq尾指针、cq头指针、门铃寄存器的操作

主机和ssd的通信过程整体如下：

1. 主机向ssd发送读写命令：主机往sq里写入nvme_cmd，移动sq尾指针，并把移动后的sq尾指针写入sq的门铃寄存器
2. ssd感知到sq的门铃寄存器更新后，会读取sq里的读写命令，进行处理，类似上文在介绍prp时的描述
3. ssd处理完读写命令后，会把处理结果写入cq
4. 主机在cq中获取读写命令的结果，然后移动cq头指针，把移动后的cq头指针写入cq的门铃寄存器，通知ssd该cq条目已被处理

GPU和CPU共享NVME SSD

CPU和GPU是默认不能共享同一个nvme ssd的，因为NVME协议规定nvme ssd只能有一对admin queue。如果CPU和GPU对同一个ssd各有一对admin queue进行管理，那很明显会产生冲突。

Geminifs的创新点之一在于，敏锐地发现在GPU读写ssd的场景中，GPU可以只有io queue，而不需要有admin queue。即admin queue仍然只由CPU管理，这样一个ssd仍然只有一个admin queue，CPU和GPU可以共享同一个nvme ssd。

PCIE P2P以及NVME SSD地址、内存地址、显存地址之间的映射

GPU绕过CPU直接读写ssd，底层的物理技术是PCIE P2P，这是传统DMA技术的扩展：

- 传统DMA：外设不需要CPU处理，直接读写内存
- PCIE P2P：外设之间不需要CPU处理，互相读写数据

GPU和SSD之间要进行PCIE P2P通信，需要互相知道对方的地址，有两种实现方式：

1. 互相知道对方的物理地址
2. 通过IOMMU，进行虚拟地址到物理地址的映射。IOMMU的功能和工作方式可类比MMU。

IOMMU在linux 的PCIE P2P场景中还比较脆弱（Bam的文档里提到），且走IOMMU会有性能损耗，所以Bam和Geminifs都关闭了IOMMU，选择的是直接通过物理地址的方式。

Geminifs中让GPU知道ssd的队列和门铃寄存器的地址的整体流程如下：

1. Controller mmap /dev/ssnvme0,内核模块把ssd的PCI地址映射进虚拟内存，返回void* mm_ptr，供用户态访问
2. Controller调cudaHostRegister，对void* mm_ptr锁页，让GPU可以访问这段固定内存
3. Controller 计算每对sq/cq的门铃寄存器映射在虚拟内存的地址：通过调CQ_DBL/SQ_DBL宏，基于ssd PCI地址映射到虚拟内存的mm_ptr、队列编号、步长计算获得。
4. Controller迭代处理每个队列，获取cq和sq的门铃寄存器在GPU端的映射指针，拷贝在显存中，让GPU后续可以通过这些地址直接对ssd的队列、门铃寄存器发起访问

Geminifs中让ssd知道gpu的显存地址的整体流程如下：



1. Geminifs ioctl /dev/ssnvme0, 自定义内核模块中会调用GPU显卡驱动提供的函数，得到tensor的显存物理地址，并建立DMA映射，以支持这段显存地址在后续被ssd通过DMA方式访问
2. Geminifs 对tensor切片（以方便GPU线程并发执行IO读写），对每个tensor切片都计算对应的物理显存地址，填充在prp相关字段中
3. Geminifs 对ssd发起读写时，读写切片内容的prp相关字段会被包含在nvme_cmd里，并且Geminifs会基于显存里的sq/cq的门铃寄存器地址，通过内联汇编往门铃寄存器里写入更新后的sq尾指针/cq头指针地址。



对文件的管理以及文件元数据的使用

1) NVMEFile、GPUFile

Geminifs中有NVMEFile、GPUFile 这2个概念：

- NVMEFile：在nvme ssd上的物理文件，在磁盘和内存中
- GPUFile：Geminifs专有概念，GPU视角所看到和管理的文件，在显存中

Geminifs支持多个nvme ssd,GPUFile和NVMEFile的数据内容映射关系取决于所使用的ssd数量：

- 当只使用1个ssd时，GPUFile和NVMEFile的数据内容相同，只是数据所在的位置不同。
- 当使用多个ssd时，GPUFile和NVMEFile类似RAID0，一个GPUFile，被条带化非冗余地存储在多个NVMEFile上。

2) 文件元数据的管理和使用：

Geminifs的文件是有专门的格式的：在文件的起始头部区域，有专门的自包含元数据，内容如下：

geminifs_hdr：文件头部的自描述元数据结构	
+ magic_num:uint64	// 魔数，用于确认文件类型是geminifs文件
+ first_block_base:uint64_t	// 文件头部的元数据字节总数
+ virtual_space_size:uint64_t	// 文件总字节数
+ fd: int	// 文件被打开后的fd
+ extents: gemini_fiemap_extent[]	// 描述每个extent的长度、在文件的起始逻辑偏移、在磁盘上的起始物理偏移、flags
+ extent_count: uint8_t	// extent的数量

文件系统元数据在CPU上进行管理并和GPU共享，GPU访问的文件都可以由CPU管理（如增删改），GPU可以直接提交文件IO请求。

Geminifs在创建文件时，CPU会给文件预分配文件元数据的体积，并在创建完文件之后，在文件头部写入上述元数据。

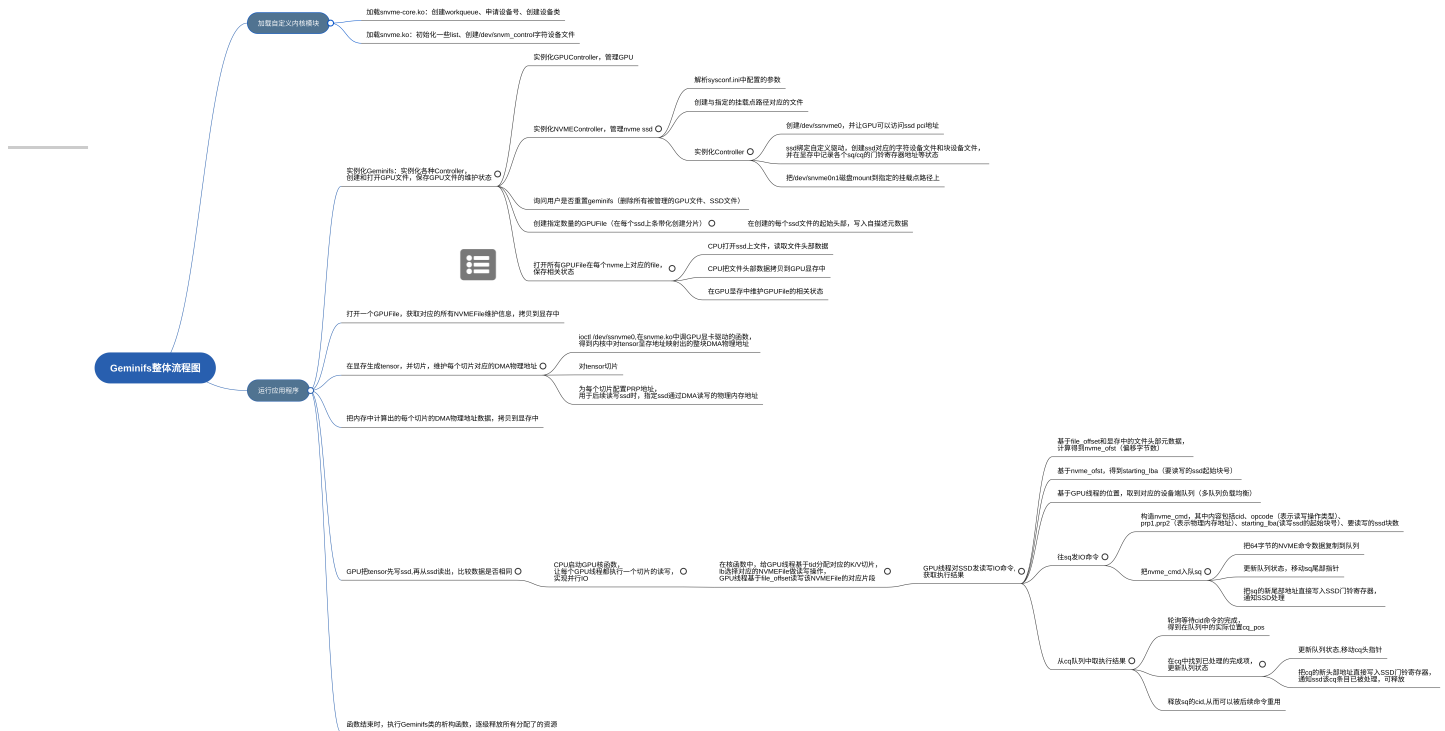
GPU对于文件元数据的使用方式：

1. GPU在读写文件时，需要CPU先打开文件，读取文件头部的元数据，然后拷贝到GPU显存。
2. GPU基于从CPU内存拷贝到显存的文件头部元数据（磁盘块大小、extent的数量和信息），在给nvme ssd发读写IO命令时，基于要读写的file_offset，转化到nvme_offset、startLBA（读写ssd的起始逻辑块号），把startLBA和要读写的块数填入构造的nvme_cmd中，即可指示ssd要读写ssd的数据位置。

对于传统文件，是没有geminifs需要的元数据的，geminifs不能直接读取，可考虑在传统文件之外，再创建一个辅助文件，存有该文件的geminifs格式的元数据，这样geminifs在读写该数据文件时，先通过读取辅助文件的元数据，是可以读写传统文件的。

Geminifs的具体实现

整体流程



创建和使用的设备文件

Geminifs在运行过程中，会创建多个设备文件，如下图，这些设备文件分别有对应的作用，也是观察理解Geminifs的比较直观的切入口。

```

pk1@powerleader:~$ ll /dev/*nvm*
crw----- 1 root root 240, 1 Sep 28 01:16 /dev/nvme1
brw-rw---- 1 root disk 259, 2 Sep 28 01:16 /dev/nvme1n1
crw----- 1 root root 240, 2 Sep 28 01:16 /dev/nvme2
brw-rw---- 1 root disk 259, 1 Sep 28 01:16 /dev/nvme2n1
crw----- 1 root root 496, 64 Sep 28 03:04 /dev/snvme_control
crw----- 1 root root 499, 0 Sep 30 02:19 /dev/snvme0
brw-rw---- 1 root disk 259, 0 Sep 30 02:19 /dev/snvme0n1
crw----- 1 root root 496, 0 Sep 30 02:19 /dev/ssnvme0

pk1@powerleader:~$ ll /sys/block/*nvm*
lrwxrwxrwx 1 root root 0 Sep 28 01:16 /sys/block/nvme1n1 -> ../devices/pci0000:b0/0000:b0:02.0/0000:b1:00.0/nvme/nvme1/nvme1n1/
lrwxrwxrwx 1 root root 0 Sep 28 01:16 /sys/block/nvme2n1 -> ../devices/pci0000:e2/0000:e2:02.0/0000:e3:00.0/nvme/nvme2/nvme2n1/
lrwxrwxrwx 1 root root 0 Sep 30 02:19 /sys/block/snvme0n1 -> ../devices/pci0000:30/0000:30:02.0/0000:31:00.0/snvme/snvme0/snvme0n1/

pk1@powerleader:~$ lsmod |grep nvme
snvme                196608 4
snvme_core            106496 2 snvme
nvme                  45056 1
nvme core            126976 4 snvme,nvme,snvme_core

```

图.运行TestForGeminiFS二进制文件时的状态

设备文件	文件类型	对应字符驱动函数	文件作用	创建时间及方式
/dev/snvme_control	字符文件设备	snvm_fops	ioctl fd时，基于传入的type类型执行对应操作，见下表	加载snvme.ko时创建
/dev/ssnvme0	字符文件设备	snvm_dev_fops	1. mmap fd时，把ssd的pci地址映射进虚拟内存 2. ioctl fd时，基于传入的type类型执行对应操作，见下表	Controller构造函数中，执行nvm_chrdev_create时 ioctl /dev/snvme_control,把ssd pci地址作为参数传入
/dev/snvme0	字符文件设备	nvme_dev_fops	nvme ssd的IO命令控制文件	Controller构造函数中，把ssd和自定义驱动绑定时， ioctl /dev/snvme_control,把ssd pci地址作为参数传入， 内核模块执行pci_register_driver(&snvme_driver)， 在nvme_probe函数的snvme_init_ctrl子函数
/dev/snvme0n1	块设备	无	作为块设备使用，如格式化文件系统、挂载	Controller构造函数中，把ssd和自定义驱动绑定时， ioctl /dev/snvme_control,把ssd pci地址作为参数传入， 内核模块执行pci_register_driver(&snvme_driver)， 在nvme_probe函数的末尾

ioctl /dev/snvme_control传入的部分命令参数类型	snvme.ko在snvm_fops函数中执行的操作内容
SNVM_DEVICE_BIND	先把nvme ssd与系统默认驱动解绑，然后执行pci_register_driver(&snvme_driver)，来把nvme ssd与自定义驱动绑定
SNVM_CHRDEV_CREATE	创建字符设备文件（如/dev/ssnvme0），关联驱动函数snvm_dev_fops
SNVM_DEVICE_UNBIND	将nvme ssd和自定义驱动解绑
SNVM_CALCULATE_PCIDISTANCE	计算两个PCI设备的距离

ioctl /dev/ssnvme0传入的部分命令参数类型	snvme.ko在snvm_dev_fops函数中执行的操作内容
NVM_MAP_DEVICE_MEMORY NVM_MAP_DEVICE_QUEUE_MEMORY	将用户态cuda malloc 分配的地址pin住并得到dma地址返回用户态
NVM_SET_IOQ_NUM	设置内核态数据结构ctrl的ioq_num、on_host
NVM_GET_DEV_INFO	获取磁盘的namespace信息





▼ 类图







▼ 具体运行过程

运行过程为先加载自定义内核模块snvme-core.ko和snvme.ko，然后再运行demo（TestForGeminiFS），具体过程如下

▼ 先加载自定义内核驱动模块

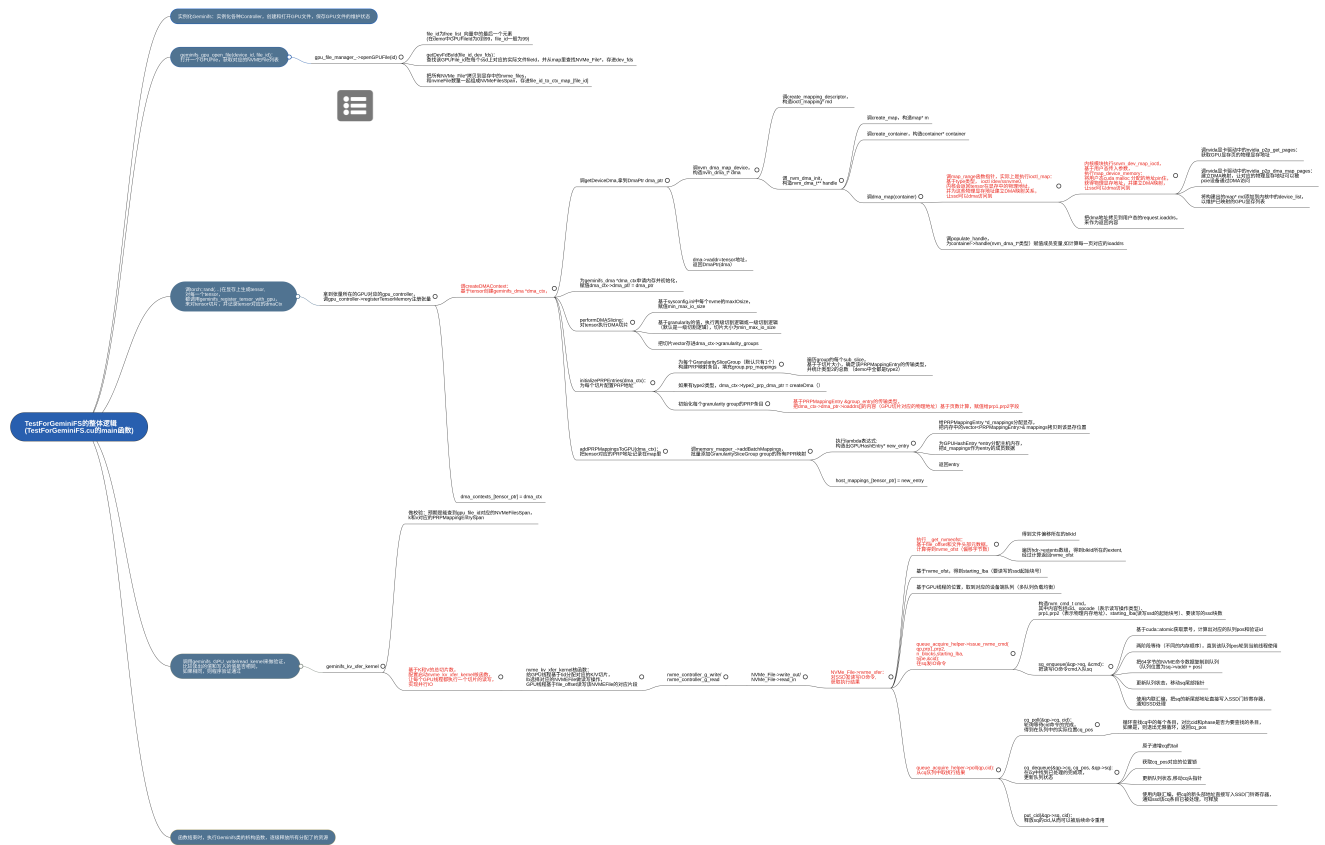
先加载snvme-core.ko，再加载snvme.ko



▼ 再运行应用程序

TestForGeminiFS的整体逻辑

Geminifs项目当前的可用demo为examples/test_geminifs,该目录下可编译出TestForGeminiFS二进制程序，该demo的整体运行逻辑如下：



Geminifs的构造函数





▼ Geminifs和VLLM的结合

思路：

—— Geminifs的优点在于对nvme ssd的高性能读写，可以和VLLM对KVCache、权重的offload结合起来。

VLLM通过offload和调度技术，把显存中暂时不需要用到的冷的KVCache、权重等数据，offload到CPU DRAM、NVME SSD，来节省所使用的GPU显存，实现对显存的高效利用，提高吞吐；当冷数据需要用到时，VLLM再从ssd读出数据。



可参考MoonCake。

实现：

Geminifs是C++项目，VLLM是python项目，可通过pybind11，把Geminifs的C++接口封装为python模块，供VLLM调用。

可以先实现基础原型，走通VLLM调用Geminifs的过程，验证端到端可行性。

无标签